What is an abstract class, and what makes a method "abstract" in Python?

An abstract class is a class that uses @abstractmethod + ABC to imply that this certain method must be overriden by a subclass and if it isn't overriden then the object simply can't be created.

Abstraction is achieved by using the @abstractmethod decorator + inheriting from ABC , which together block instantiation until the method is overridden.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    def describe(self):
        print("I am a shape")

class Circle(Shape):
    pass

c = Circle()
```

What is partial abstraction vs full abstraction?

Partial = some methods abstract, some concrete (ex: `Shape`).

Full = every method is abstract, parent provides zero real implementation.

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    def describe(self):
        print("I am a shape")

class Circle(Shape):
    def area(self):
        print("Circle area")

c = Circle()
c.describe()
c.area()
```

Does Python have a dedicated interface keyword like Java or C++? If not, how do Python developers achieve the same effect?

Java/C++ "interface" is basically a contract that says "any class implementing me must provide these exact methods, and I provide zero actual code myself , just

method names." That's the equivalent of using full abstraction and @abstractmethod in python.

What happens if you try to instantiate a class that has unimplemented abstract methods? "TYPE ERROR"

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    def describe(self):
        print("I am a shape")

class Circle(Shape):
    pass

c = Circle()
```

```
Traceback (most recent call last):
  File "C:\Users\Mido Computer\PycharmProjects\WelcomeScreen\question1.py", line 14, in
<module>
    c = Circle()
TypeError: Can't instantiate abstract class Circle without an implementation for abstract
method 'area'

Process finished with exit code 1
```

Diff between Duck Typing and Polymorphism:

Polymorphism is inheritance through parent while duck typing requires no common parent, just method calling.

What is duck typing, and where does the name come from?

The idea that Python cares about an object's behavior (does it have the method being called?) rather than its declared type. Name comes from "if it walks like a duck and quacks like a duck, it's treated as a duck."

```
class Duck:
    def sound(self):
        print("Quack")

class Dog:
    def sound(self):
        print("Woof")

def make_it_sound(animal):
    animal.sound()

make_it_sound(Duck())
make_it_sound(Dog())
```

How does duck typing let different objects be used interchangeably without inheriting from a common parent?

As shown with `Duck` and `Dog` , completely unrelated classes all worked with `make_it_sound()` because Python just calls `.sound()` and doesn't check the object's class/ancestry at all.

Give an example of duck typing being used instead of strict type-checking.

Strict-type checking:

```
def make_it_sound_strict(animal):
    if isinstance(animal, Duck) or isinstance(animal, Dog):
        animal.sound()
    else:
        print("Not a recognized animal type!")
```

duck-typing:

```
class Duck:
    def sound(self):
        print("Quack")

class Dog:
    def sound(self):
        print("Woof")

def make_it_sound(animal):
    animal.sound()

make_it_sound(Duck())
make_it_sound(Dog())

class Robot:
    def sound(self):
        print("Beep boop")

make_it_sound(Robot())
```

Even though `Robot` has zero relationship to `Duck` or `Dog` as in no shared parent, no inheritance at all. Python only cares that `Robot` has a `.sound()` method it can call.